



THE YENEPLOYA INSTITUTE OF ARTS SCIENCE
COMMERCE AND MANAGEMENT

PROJECT-LOW LEVEL DESIGN

On

Honeypot Network for Healthcare Threat Intel

Student Name:

Mohammed Rasmil P.V

23BCCE116

Industry Mentor::

Mr.Shashank Korram

Guided by:

Ms.Munisah

TABLE OF CONTENT :

1. Introduction	3
1.1 Purpose and Scope	3
1.2 Project Executive Summary	4
1.3 Constraints	4
1.4 Document Organization	5
2. System Overview	5
3. Module Design	6
3.1 Socket Listener Module	6
3.2 Connection Handler	6
Module 3.3 Logger Module	6
3.4 Configuration Module	6
3.5 ResponseModule	6
4. Data Flow Design	7
5. Database Design	7
5.1 Connection Log Table	7
5.2 Alert Table	7
5.3 ConfigTable	8
6. User Interface Design	8
7. Security Implementation	8
8. Testing and Validation	8
9. Project References	9
10. Glossary	9
11. Front-end	10
12. Back-end	11

1. INTRODUCTION

1.1 Purpose and Scope

Purpose:

This document provides a detailed low-level design of a Honeypot Network system tailored for a Healthcare environment. The goal is to design a system that detects and logs unauthorised network access attempts, mimicking a real healthcare server to gather threat intelligence.

The system focuses on maintaining the integrity, confidentiality, and availability of the healthcare network by identifying and logging suspicious connection attempts in real time.

Scope:

The system covers:

- Port-based TCP honeypot listener
- IP address and timestamp logging of all connection attempts
- Fake response generation to mislead attackers
- Persistent log storage for security audit
- Console monitoring of live threat events

In Scope:

- Python socket-based listener on port 8888
- Append-mode file logging
- Fake HTTP 403 Access Denied response
- Graceful shutdown on keyboard interrupt
- Log review by security administrator

Out of Scope

- Realpatient data handling
- Integration with hospital management systems
- GUI-based monitoring dashboard Multi-threaded or distributed deployment

1.2 Project Executive Summary

Modern healthcare networks store highly sensitive patient data including medical records, diagnostics, and personal information. Ensuring network security is critical to prevent unauthorised access, data breaches, and regulatory violations.

This project introduces a Honeypot-based threat intelligence system that:

- Listens for TCP connection attempts on a designated port
- Logs attacker IP address and connection timestamp automatically
- Sends a fake Access Denied response to deceive the intruder
- Maintains a persistent log file for forensic and audit purposes
- Operates continuously until manually stopped

The system is implemented using:

- Language: Python 3.x
- Library: Socket (built-in)
- Storage: Local log file (honeypot.log)

This solution enhances healthcare network security awareness and enables administrators to monitor threat patterns.

1.3 Constraints

Technical Constraints

- Single-threaded design handles one connection at a time
- Python socket library provides no built-in encryption Log
- storage is file-based with no database backend
- No real-time alerting or notification system

Security Constraints

- Must be run in an isolated or sandboxed environment
- Attacker data must be handled as per data protection guidelines
- Port 8888 must not be used by any real healthcare service

Operational Constraints

- Designed for academic and demonstration purposes
- Limited to local network deployment in lab environment
- Requires Python 3.6 or above on the host machine

1.4 Document Organization

The document is structured into:

1. Introduction
2. System Overview
3. Module Design
4. Data Flow
5. Database Structure
6. Interface Design
7. Security Implementation
8. Testing Strategy

2. System Overview

The system is designed using a

The system is designed using a **command-line architecture**:

- The honeypot runs as a background Python process
- Incoming TCP connections are accepted by the socket listener
- Each connection triggers logging and fake response routines

The system ensures that every connection attempt is logged before any response is issued.

3. Module Design

3.1. Socket Listener Module

- Bindsto host 0.0.0.0 onport 8888
- Creates TCP socket using AF_INET and SOCK_STREAM
- Continuously accepts incoming connections

3.2. Connection Handler Module

- Extracts attacker IP address andsource port from connection metadata
- Passes connection details to logger and response modules
- Closes each client socket after response is sent

3.3. Logger Module

- Records IPaddress and timestamp to honeypot.log
- Writes simultaneously to console for live monitoring
- Uses append mode to preserve historical records

3.4. Configuration Module

- Storesall constants:HOST, PORT, LOG_FILE, FAKE_RESPONSE
- Centralises parameters for easy modification
- Eliminates hardcoded values from core logic

3.5. Response Module

- Generates fake HTTP 403 Forbidden response
- Sends response to attacker via client socket
- Mimics a real healthcare server to deceive intruder

4. Data Flow Design

- i. Attacker or scanner initiates TCP connection to port 8888
- ii. `server_socket.accept()` receives the connection
- iii. Attacker IP address and source port are extracted
- iv. Logger module writes event to `honeypot.log` and console
- v. Response module sends fake HTTP 403 Access Denied message
- vi. Client socket is closed
- vii. Listener returns to `accept()` for the next connection
- viii. All events are stored persistently in the log file

5. Database Design

Tables Used

Connection Log Table

- `log_id`
- `ip_address`
- `source_port`
- `timestamp`
- `response_sent`

Alert Table

- `alert_id`
- `log_id`
- `alert_type`
- `severity`

Config Table

- config_id
- host
- port
- log_file_path

6. User Interface Design

The system includes:

- Terminal / Console — live display of connection events
- Log File Viewer — review of historical connection records
- Config File — editable constants for host, port, and response

The interface is designed to be minimal, functional, and easy for security administrators to use.

7. Security Implementation

- No real data is exposed — all responses are fake
- Attacker IP is logged but never processed or executed System
- runs in isolation to prevent lateral movement
- Log files are write-only to prevent tampering via the process
- Port 8888 is only used for the honeypot and nothing else

8. Testing and Validation

- Functional testing of TCP connection acceptance
- Log file verification after each connection
- Fake response content validation using telnet or netcat
- Graceful shutdown testing using keyboard interrupt (Ctrl+C)

9. Project References

- Python Official Documentation — socket module
- Python Official Documentation — logging module
- OWASP Honeypot Project Guidelines
- VS Code

10. Glossary

- Honeypot: A security decoy trap designed to detect unauthorised access
- TCP: Transmission Control Protocol — connection-oriented network protocol
- Socket: Software endpoint for network communication
- IP Address: Unique network identifier of a device
- Port: Logical endpoint for network communication (0–65535)
- Threat Intel: Information used to understand potential cyber attacks

FRONT-END

MED-SHIELD HONEYPOT

● ACTIVE THREAT SCANNER

TOTAL THREATS DETECTED
7

UNIQUE ENTITY IPS
3

HONEYPOT STATUS
● **ONLINE (PORT 8888)**

INTEL FEED

REFRESHING EVERY 2 SECONDS

TIMESTAMP	IDENTIFIER / SOURCE IP	IDENTIFIED USER AGENT	ACTION
2026-04-27 11:14:21	192.168.1.6	TCP Raw Connection	DENIED
2026-04-27 09:37:27	192.168.1.6	TCP Raw Connection	DENIED
2026-04-27 09:32:28	192.168.1.6	TCP Raw Connection	DENIED
2026-04-27 09:32:14	192.168.1.6	TCP Raw Connection	DENIED
2026-04-27 09:32:02	192.168.1.6	TCP Raw Connection	DENIED
2026-04-13 16:10:38	192.168.1.37	TCP Raw Connection	DENIED
2026-04-13 16:10:31	192.168.1.38	TCP Raw Connection	DENIED

© 2026 Healthcare Threat Intelligence. Unauthorized access is monitored.

BACK-END – PYTHON

```

import socket
import threading
from datetime import datetime
import json
import os
from flask import Flask, render_template, jsonify, send_from_directory
from flask_cors import CORS

# --- CONFIGURATION ---
HOST = '0.0.0.0'
TRAP_PORT = 8888
DASH_PORT = 5001
LOG_FILE = 'logs/threat_intel.json'
MESSAGE = b"Access Denied: You are not authorized to view this resource.\n"

app = Flask(__name__, template_folder='templates', static_folder='static')
CORS(app)

# Cache for real-time dashboard updates without reading file every time
intel_logs = []

def load_logs():
    """Loads existing logs from the JSON file."""
    global intel_logs
    if os.path.exists(LOG_FILE):
        try:
            with open(LOG_FILE, "r") as f:
                intel_logs = json.load(f)
        except (json.JSONDecodeError, ValueError):
            intel_logs = []

def save_log(entry):
    """Saves a new log entry to the JSON file and cache."""
    global intel_logs
    intel_logs.append(entry)

if len(intel_logs) > 100:
    intel_logs = intel_logs[-100:]

os.makedirs(os.path.dirname(LOG_FILE), exist_ok=True)
with open(LOG_FILE, "w") as f:
    json.dump(intel_logs, f, indent=4)

log_event(ip_address, user_agent=None):
    """Formats and records a connection attempt."""
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    entry = {
        "timestamp": timestamp,
        "ip": ip_address,
        "ua": user_agent or "TCP Raw Connection",
        "status": "BLOCKED"
    }
    save_log(entry)
    print(f"[*] THREAT DETECTED: {ip_address} at {timestamp}")

-- THE TRAP (SOCKET LISTENER) --
start_trap():
    """Background TCP socket listener to capture any connection (not just HTTP)."""
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    try:
        server.bind((HOST, TRAP_PORT))
        server.listen(5)
        print(f"[*] Honeypot Trap active on port {TRAP_PORT}")

        while True:
            client, addr = server.accept()
            # Handle the detection
            log_event(addr[0])

```

```
    client.sendall(MESSAGE)
except:
    pass
finally:
    client.close()
except Exception as e:
    print(f"[!] Trap Error: {e}")
finally:
    server.close()
```

```
# --- THE WEB DASHBOARD (FLASK) ---
@app.route('/')
def dashboard():
    """Serves the main threat intel dashboard."""
    return render_template('index.html')

@app.route('/api/logs')
def get_logs():
    """API endpoint for live dashboard data."""
    return jsonify(intel_logs)

@app.route('/api/stats')
def get_stats():
    """API endpoint for dashboard counters."""
    unique_ips = len(set(log['ip'] for log in intel_logs))
    total_attempts = len(intel_logs)
    return jsonify({
        "total": total_attempts,
        "unique": unique_ips,
```

```
if __name__ == "__main__":
    # Pre-load logs
    load_logs()

    # Start the TCP Trap in a background thread
    trap_thread = threading.Thread(target=start_trap, daemon=True)
    trap_thread.start()

    # Start the Admin Dashboard on port 5000
    print(f"[*] Launching Admin Dashboard at http://localhost:{DASH_PORT}")
    app.run(host=HOST, port=DASH_PORT, debug=False)
```